

Festival Planner

Vektorsuche und LLM

Roadmap

Phase 1 - Vektorsuche

- 1 Frontend-Zielbild
- 2 Requirement
- 3 Suchinhalt festlegen
- 4 Domain Model dokumentieren
- 5 Embedding-Modell ausprobieren
- 6 pgvector aktivieren
- 7 embedding-Spalte technisch anlegen
- 8 Embeddings erzeugen + speichern
- 9 Vektorsuche direkt testen
- 10 Backend / SQLAlchemy-Suche
- 11 Search-API
- 12 Frontend anbinden
- 13 Tests + Review

Plan

Phase 1 - Semantische Vektorsuche (ohne LLM)

Ziel: Acts nicht nur über exakte Begriffe finden, sondern nach Bedeutung (z. B. „ruhige Musik am Abend“). Ergebnis ist eine nach Ähnlichkeit sortierte Liste passender Acts.

Suchanfrage → Embedding-Modell → Query-Vektor → PostgreSQL/pgvector → passende Acts

Geplant: durchsuchbare Textfelder im Domain Model festlegen, pgvector in PostgreSQL aktivieren, Embeddings erzeugen und speichern, Ähnlichkeitssuche im Backend als FastAPI-Endpoint bereitstellen und im Frontend einbinden – jeweils mit Tests und Review.

Phase 2 - LLM Integration

Die Treffer aus Phase 1 werden einem LLM als Kontext übergeben, das daraus eine Antwort auf Fragen in natürlicher Sprache formuliert.

Suchanfrage → Vektorsuche → passende Acts → LLM-Kontext → generierte Antwort

Die Vektorsuche bleibt dabei die Retrieval-Schicht; das LLM ersetzt die Datenbanksuche nicht und darf keine Festivalinformationen erfinden, die nicht in den gefundenen Daten stehen.

Output

Phase 1 - Vector Search / Retrieval Layer

Nutzer fragt:

„Ich suche heute Abend etwas Ruhiges, eher atmosphärisch.“

Die Anwendung macht daraus ein Embedding und findet ähnliche Acts.

Das Ergebnis könnte direkt so aussehen:

1. Neon Echo
Genre: Ambient Electronic
Bühne: Lake Stage
Zeit: 20:30
Distanz: 0.08
2. Luna Fields
Genre: Indie Folk
Bühne: Forest Stage
Zeit: 21:00
Distanz: 0.14
3. Dream Waves
Genre: Dream Pop
Bühne: Main Stage
Zeit: 22:00
Distanz: 0.19

Das System sagt also im Kern nur:

„Diese Datensätze passen semantisch am besten zu deiner Anfrage.“

Keine eigentliche sprachliche Interpretation oder Empfehlung durch ein LLM.

Phase 2 - LLM Integration (Generation Layer)

Die gleichen drei Treffer werden jetzt zusätzlich an ein LLM gegeben, zusammen mit der ursprünglichen Frage.

Das Ergebnis könnte dann sein:

„Wenn du etwas Ruhiges und Atmosphärisches suchst, ist Neon Echo vermutlich die passendste Wahl. Luna Fields ist akustischer und weniger elektronisch, während Dream Waves etwas poppiger klingt. Wenn du möglichst entspannt starten möchtest, würde ich mit Neon Echo um 20:30 beginnen.“

Jetzt passiert mehr:

Nutzerfrage → Retrieval → gefundene Daten → LLM → formulierte Antwort

Der entscheidende Unterschied ist also:

Phase 1 liefert Treffer.

Phase 2 erklärt, kombiniert und formuliert auf Basis dieser Treffer.

oder einfach:

Phase 1: "Hier sind passende Acts."

Phase 2: "Hier sind passende Acts – und so passen sie zu deiner Frage."

Plan

Phase 1 - Semantische Vektorsuche ohne LLM

Ziel:

Hinweis

```
Suchtext
  ↓
Embedding-Modell
  ↓
Query-Vektor
  ↓
PostgreSQL + pgvector
  ↓
ähnlichste Artists
  ↓
zugehörige Acts
  ↓
Trefferliste
```

Beispiel: Der Festival Planner bekommt eine natürliche Suche wie:

„Ich suche ruhige elektronische Musik am Abend.“

Die Anwendung liefert dazu passende Acts. Noch **kein LLM**, sondern reine semantische Suche.

Ergebnis von Phase 1:

Eine Trefferliste mit passenden Acts, z. B. Neon Echo, Luna Fields, Dream Waves.

Entwicklungsreihenfolge (Phase 1)

1. Frontend-Zielbild bauen

- Suchfeld ergänzen
- Button „Suchen“
- Bereich für Suchergebnisse
- zunächst mit Mock-Daten
- Beispielquery sichtbar machen: „ruhige elektronische Musik“

Ziel: Kunde/Stakeholder sollen sehen, wo die Reise hingeht.

2. Requirement „Semantische Suche“ ergänzen

Zum Beispiel:

M7 - Semantische Suche:

Nutzer können Acts über natürlich formulierte Suchanfragen finden.
Die Ergebnisse werden nach semantischer Ähnlichkeit sortiert.

Noch kein LLM.

3. Festlegen, was semantisch durchsucht wird

Aktuell reichen `Artist`, `Stage`, `starts_at`, `ends_at` noch nicht für gute semantische Suche.

Deshalb Suchinformationen ergänzen, etwa:

```
Artist
- name
- genre
- description
```

Für das Embedding wird daraus z. B.:

Neon Echo. Genre: Electronic Ambient.
Atmosphärische elektronische Musik mit ruhigen Synthesizer-Flächen.

4. Domain Model erweitern

Sinnvoll wäre zunächst:

```
Artist
├── id
├── name
├── genre
└── description
```

Dabei bewusst klären:

Was ist fachlicher Inhalt und was ist nur technische Repräsentation?

genre und description sind fachliche Daten.

5. Embedding-Modell separat ausprobieren

Embedding Modell installieren

Embedding-Modell: lokal

sentence-transformers/all-MiniLM-L6-v2 , **384 Dimensionen**. Keine API-Kosten, keine Keys, und die Embedding-Erzeugung bleibt sichtbar und nachvollziehbar.

```
pip install sentence-transformers
```

Bei sentence-transformers/all-MiniLM-L6-v2 liefert das Modell standardmäßig immer Vektoren der Länge: 384 -

Andere Embedding-Modelle haben andere Dimensionen, zum Beispiel 768, 1024 oder noch mehr.

Info

Theoretisch könntest du einen 384-dimensionalen Vektor **nachträglich reduzieren**, etwa mit PCA auf 100 Dimensionen. Dann veränderst du aber die Repräsentation und musst dieselbe Transformation konsistent für alle gespeicherten Texte und Suchanfragen anwenden!

Bevor es in die App kommt, ein kleines Experiment:

```
Text
↓
Tokenizer
↓
Embedding-Modell
↓
Vektor mit 384 Zahlen
```

Modell beispielsweise:

```
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
```

Damit wird sichtbar, dass das Embedding-Modell kein Textsuchsystem ist, sondern Text in einen Zahlenraum überführt.

6. pgvector in PostgreSQL aktivieren

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Danach bekommt das entsprechende Modell ein Vector-Feld, z. B. bei MiniLM:

```
vector(384)
```

Danach kennt PostgreSQL zusätzlich unter anderem den Typ: *vector*

7. `Artist.embedding` als `vector(384)` in DB/SQLAlchemy ergänzen (embedding-Spalt)

8. Embeddings für Festival-Daten erzeugen + speichern

Beim Seed oder in einem separaten Schritt:

```
Artist + Genre + Description
↓
Embedding-Modell
↓
Vektor
↓
PostgreSQL
```

Danach enthält jeder relevante Datensatz zusätzlich sein Embedding.

9. Vektorsuche zunächst direkt testen

Eine Suchanfrage wie:

```
"ruhige elektronische Musik"
```

wird ebenfalls eingebettet.

Dann:

```
Query
  ↓
Query-Embedding
  ↓
Vergleich mit gespeicherten Embeddings
  ↓
kleinste Distanz
  ↓
beste Treffer
```

Zum Beispiel:

Neon Echo	0.08
Dream Waves	0.15
Iron Wolves	0.71

10. Vektorsuche in SQLAlchemy integrieren (Backend)

Die Suche wandert in die bestehende Architektur:

```
routes.py
  ↓
crud.py
  ↓
SQLAlchemy
  ↓
PostgreSQL + pgvector
```

Zum Beispiel fachlich als:

```
semantic_search(query_embedding, limit=3)
```

11. Search-API ergänzen

Etwa:

```
GET /api/search?q=ruhige elektronische musik
```

Intern:

```
Suchtext
↓
Embedding
↓
pgvector
↓
Top-k Acts
↓
JSON
```

-> Test in Swagger

12. Frontend mit echter Suche verbinden

Jetzt ersetzt die reale API die Mock-Daten.

Ergebnis:

```
Suche:
"ruhige elektronische Musik"

Neon Echo
Ambient Electronic
Lake Stage · 20:30

Dream Waves
Dream Pop
Forest Stage · 21:00
```

13. Tests und Review

Prüfen:

- Embeddings werden erzeugt
- Vector-Feld ist korrekt
- Suchendpoint funktioniert
- Top-k wird eingehalten
- leere Query wird behandelt
- Suchergebnisse sind stabil
- bestehende Festival-Funktionen bleiben intakt

Entwicklungsreihenfolge (Phase 2)

```
Suchanfrage
↓
Embedding
↓
pgvector-Suche
↓
Top-5 Artists + Acts
↓
Kontext für LLM bauen
↓
LLM
↓
formulierte Antwort
```

1. Requirement für LLM-Antwort ergänzen

Zuerst beschreibst du fachlich, **was die generierte Antwort leisten soll**.

Zum Beispiel:

```
C12:
Die Anwendung soll zu einer semantischen Suchanfrage zusätzlich
eine natürlichsprachliche Antwort erzeugen.

F12:
Die Antwort darf ausschließlich Informationen verwenden,
die aus den gefundenen Artists und Acts stammen.

B10:
Werden keine geeigneten Treffer gefunden, darf das LLM
keine Künstler oder Veranstaltungen erfinden.
```

Wichtig ist hier besonders die Grenze:

Das LLM sucht nicht selbst. Die Vektorsuche liefert die Fakten.

Das ist ein sehr gutes RAG-Prinzip.

2. LLM auswählen

Jetzt entscheidest du, welches Modell die Antwort erzeugt.

Für dein Projekt bietet sich ein **lokales kleines Instruct-Modell über Ollama** an, beispielsweise das von dir bereits getestete Qwen-Modell.

Wichtige Kriterien:

- lokal ausführbar
- ausreichend schnell
- Instruct/Chat-Modell
- deutschsprachige Antworten brauchbar
- geringer RAM-Verbrauch
- reproduzierbar für die Teilnehmer

Danach sollte die technische Entscheidung dokumentiert sein, etwa:

LLM:
Qwen3-4B-Instruct

Runtime:
Ollama

Aufgabe:
Formulierung einer Antwort aus Retrieval-Ergebnissen

3. Festlegen, welche Retrieval-Daten an das LLM gehen

Hier entscheidest du, **welche Daten aus Phase 1 in den Kontext kommen**.

Hinweis

Für den Festival Planner würde ich nicht den kompletten Datenbankdatensatz übertragen, sondern nur relevante Felder:

Artist

- name
- genre
- description

Act

- stage
- starts_at
- ends_at

Zum Beispiel:

Artist:
Neon Echo

Genre:
Synth Pop

Beschreibung:
Elektronischer Pop mit atmosphärischen Synthesizern.

Auftritt:
Zeltbühne
22.09.2026
15:30–17:00

Hinweis

Die Embeddings selbst gehen **nicht** an das LLM.

4. Kontextformat definieren

Jetzt legst du fest, wie die Retrieval-Treffer in Text umgewandelt werden.

Beispiel:

Gefundene Acts:

1.
Artist: Neon Echo
Genre: Synth Pop
Beschreibung: Elektronischer Pop mit atmosphärischen Synthesizern.
Stage: Zeltbühne
Zeit: 15:30–17:00
2.
Artist: Electric Waves
Genre: Electro
Beschreibung: ...
Stage: Main Stage
Zeit: 17:30–19:00

Das klingt banal, ist aber wichtig.

Das LLM bekommt nämlich nicht die Python-Objekte:

```
Artist(...)  
Act(...)
```

sondern einen klar strukturierten Textkontext.

Dafür könnte später beispielsweise eine Funktion entstehen:

```
def build_context(results):  
    ...
```

5. System Prompt / Regeln definieren

Jetzt bekommt das Modell klare Regeln.

Der Kern könnte ungefähr so aussehen:

```
Du bist ein Assistent für einen Festivalplaner.  
  
Beantworte die Frage ausschließlich anhand des bereitgestellten Kontexts.  
  
Erfinde keine Künstler, Genres, Bühnen oder Auftrittzeiten.  
  
Wenn die Informationen nicht ausreichen, sage dies ausdrücklich.  
  
Antworte kurz und auf Deutsch.
```

Zusätzlich kannst du festlegen:

- maximal 3–5 Sätze
- keine zusätzlichen Empfehlungen
- keine Informationen außerhalb des Kontexts

Hinweis

Damit entsteht ein erster Halluzinationsschutz.

6. LLM mit festem Mock-Kontext separat ausprobieren

Das LLM zunächst **ohne Datenbank und Vektorsuche** mit einem festen Beispielkontext testen.

Dabei prüfen:

- folgt das Modell dem System-Prompt?
- bleibt es bei den bereitgestellten Informationen?
- funktioniert die deutsche Antwort?
- ist die Antwortzeit akzeptabel?

Ziel: Sicherstellen, dass **LLM + Prompt grundsätzlich funktionieren**, bevor die Retrieval-Ergebnisse angebunden werden.

claude-Prompt:

Teste das ausgewählte LLM isoliert mit einem festen Mock-Kontext zu 2–3 Festival-Acts und einer festen Suchfrage. Nutze den definierten System-Prompt und Ollama. Noch keine Datenbank, keine Vektorsuche und keine FastAPI-Integration. Gib Frage, Kontext und LLM-Antwort auf der Konsole aus.

7. Retrieval + LLM verbinden

Die Treffer aus der Vektorsuche werden in einen Textkontext umgewandelt und zusammen mit der Suchfrage an das LLM gegeben.

```
Query
→ Vektorsuche
→ Top-5 Artists + Acts
→ Kontext
→ LLM
→ Antwort
```

Wichtig: Das LLM bekommt nur die Retrieval-Ergebnisse, keinen direkten Datenbankzugriff.

8. LLM-Service im Backend integrieren

Die LLM-Logik in ein eigenes Modul kapseln, z. B.:

```
app/llm.py
```

Aufgabe:

```
generate_answer(question, context)
```

Darin liegen Ollama-Aufruf, Modellname, System-Prompt und Fehlerbehandlung.

9. API erweitern

Der Search-Endpoint liefert jetzt zusätzlich zur Trefferliste auch die generierte Antwort.

Zum Beispiel:

```
{
  "query": "...",
  "answer": "...",
  "results": [...]
}
```

Die Suchtreffer bleiben erhalten, damit die Grundlage der Antwort sichtbar bleibt.

10. Frontend anbinden

Den bisherigen Dummy-Text durch `answer` aus der API ersetzen.

Darunter weiterhin die eigentlichen Suchtreffer anzeigen.

Damit ist im UI klar getrennt:

```
LLM-Antwort
+
zugrunde liegende Treffer
```

11. Fehlerfälle / Halluzinationsschutz

Wichtige Regeln:

- keine Treffer → LLM nicht aufrufen
- LLM darf nur Retrieval-Daten verwenden
- keine erfundenen Künstler, Zeiten oder Bühnen
- bei LLM-Fehlern Suchtreffer trotzdem anzeigen
- Timeout / Ollama nicht erreichbar behandeln

Beispiel:

```
Keine Treffer:
"Dazu wurden keine passenden Acts gefunden."
```

12. Tests

Nicht die „Intelligenz“ des LLM testen, sondern die Integration.

Wichtige Tests:

- Kontext wird korrekt gebaut
- LLM-Service wird mit richtigem Kontext aufgerufen
- API liefert `answer` + `results`
- bei 0 Treffern wird das LLM nicht aufgerufen
- bei LLM-Fehlern bleiben die Suchtreffer verfügbar

Für Tests den LLM-Aufruf mocken.

13. Review + Dokumentation

Am Ende prüfen:

```
Query
→ Embedding
→ pgvector
→ Top-5 Artists
→ Acts
→ Kontext
→ LLM
→ Antwort
```

Dokumentieren:

Embedding-Modell
LLM + Ollama
Top 5
Mindestähnlichkeit 0,3
Kontextfelder
System-Prompt
Fehlerverhalten

Hinweis

Zentrale Architekturregel:

Das LLM formuliert nur auf Basis der Retrieval-Ergebnisse. Die eigentliche Suche bleibt Aufgabe von pgvector.